

# Trabalho Prático | DGT2821

## Programação Back-end com Java

---

*Material de **orientações** para desenvolvimento do **trabalho prático** da disciplina DGT2821*  
*Programação Back-end com Java*

---

Implementação de um cadastro de clientes em modo texto, com persistência em arquivos, baseado na tecnologia Java.

---

### *Objetivos do trabalho*

---

1. Utilizar herança e polimorfismo na definição de entidades.
2. Utilizar persistência de objetos em arquivos binários.
3. Implementar uma interface cadastral em modo texto.
4. Utilizar o controle de exceções da plataforma Java.
5. No final do projeto, o aluno terá implementado um sistema cadastral em Java, utilizando os recursos da programação orientada a objetos e a persistência em arquivos binários.

 **O trabalho deve ser feito individualmente.**

---

### *Materiais necessários para o trabalho prático*

---

- JDK e IDE NetBeans.
  - Equipamentos: Computador com JDK e NetBeans instalados.
- 

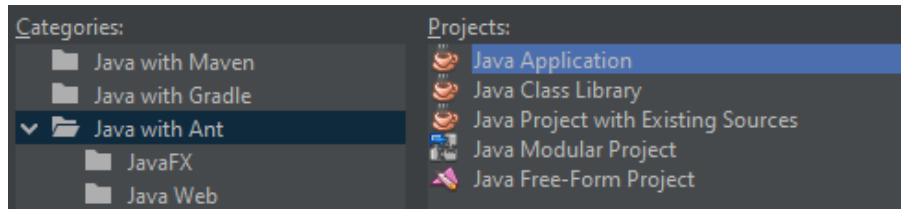
### *Desenvolvimento da prática*

---

Vamos colocar a **mão na massa**?! Siga as **instruções** abaixo para desenvolvimento deste trabalho prático.

## 👉 1º Procedimento | Criação das Entidades e Sistema de Persistência

1. Criar um projeto do tipo **Ant..Java Application** no **NetBeans**, utilizando o nome **CadastroPOO** para o projeto.



2. Criar um pacote com o nome "**model**", para as entidades e gerenciadores.

3. No pacote model criar as **entidades**, com as seguintes características:

- Classe **Pessoa**, com os campos **id** (inteiro) e **nome** (texto), método **exibir**, para impressão dos dados, construtor padrão e completo, além de getters e setters para todos os campos.
- Classe **PessoaFisica**, herdando de Pessoa, com o acréscimo dos campos **cpf** (texto) e **idade** (inteiro), método **exibir** polimórfico, construtores, getters e setters.
- Classe **PessoaJuridica**, herdando de Pessoa, com o acréscimo do campo **cnpj** (texto), método **exibir** polimórfico, construtores, getters e setters.
- Adicionar interface **Serializable** em todas as classes

4. No pacote model criar os **gerenciadores**, com as seguintes características:

- Classe **PessoaFisicaRepo**, contendo um **ArrayList** de **PessoaFisica**, nível de acesso privado, e métodos públicos **inserir**, **alterar**, **excluir**, **obter** e **obterTodos**, para gerenciamento das entidades contidas no ArrayList.
- Classe **PessoaJuridicaRepo**, com um **ArrayList** de **PessoaJuridica**, nível de acesso privado, e métodos públicos **inserir**, **alterar**, **excluir**, **obter** e **obterTodos**, para gerenciamento das entidades contidas no ArrayList .
- Em ambos os gerenciadores adicionar o método público **persistir**, com a recepção do nome do arquivo, para armazenagem dos dados no disco.
- Em ambos os gerenciadores adicionar o método público **recuperar**, com a recepção do nome do arquivo, para recuperação dos dados do disco

- Os métodos persistir e recuperar devem ecoar (**throws**) exceções
- O método obter deve retornar uma entidade a partir do id
- Os métodos inserir e alterar devem ter entidades como parâmetros
- O método excluir deve receber o id da entidade para exclusão
- O método obterTodos deve retornar o conjunto completo de entidades.

5. Alterar o método **main** da classe principal para testar os repositórios:

- Instanciar um repositório de pessoas físicas (**repo1**).
- Adicionar duas pessoas físicas, utilizando o construtor completo.
- Invocar o método de persistência em repo1, fornecendo um nome de arquivo fixo, através do código.
- Instanciar outro repositório de pessoas físicas (**repo2**).
- Invocar o método de recuperação em repo2, fornecendo o mesmo nome de arquivo utilizado anteriormente.
- Exibir os dados de todas as pessoas físicas recuperadas.
- Instanciar um repositório de pessoas jurídicas (**repo3**).
- Adicionar duas pessoas jurídicas, utilizando o construtor completo.
- Invocar o método de persistência em repo3, fornecendo um nome de arquivo fixo, através do código.
- Instanciar outro repositório de pessoas jurídicas (**repo4**).
- Invocar o método de recuperação em repo4, fornecendo o mesmo nome de arquivo utilizado anteriormente.
- Exibir os dados de todas as pessoas jurídicas recuperadas.

6. Ajustar as características para obter uma execução como a seguinte:

```
Output - CadastroPOO (run) ×
run:
Dados de Pessoa Fisica Armazenados.
Dados de Pessoa Fisica Recuperados.
Id: 1
Nome: Ana
CPF: 11111111111
Idade: 25
Id: 2
Nome: Carlos
CPF: 22222222222
Idade: 52
Dados de Pessoa Juridica Armazenados.
Dados de Pessoa Juridica Recuperados.
Id: 3
Nome: XPTO Sales
CNPJ: 33333333333333
Id: 4
Nome: XPTO Solutions
CNPJ: 44444444444444
BUILD SUCCESSFUL (total time: 1 second)
```

7. Verificar as funcionalidades solicitadas e os arquivos gerados

### Resultados esperados

1. É importante que o código seja organizado.
2. Outro ponto importante é explorar as funcionalidades oferecidas pelo NetBeans para melhoria da produtividade.
3. Nesse exercício, é esperado que o estudante demonstre as habilidades básicas para a programação Java com persistência em arquivo.

### Relatório discente de acompanhamento

Os Relatórios de Práticas deverão ser confeccionados em arquivo no formato PDF, com a Logo da Universidade, nome do Campus, nome do Curso, nome da Disciplina, número da Turma, semestre letivo, nome dos integrantes da Prática. Além disso, o projeto deve ser armazenado em um repositório no GIT e o respectivo endereço deve constar na documentação. A documentação do projeto deve conter:

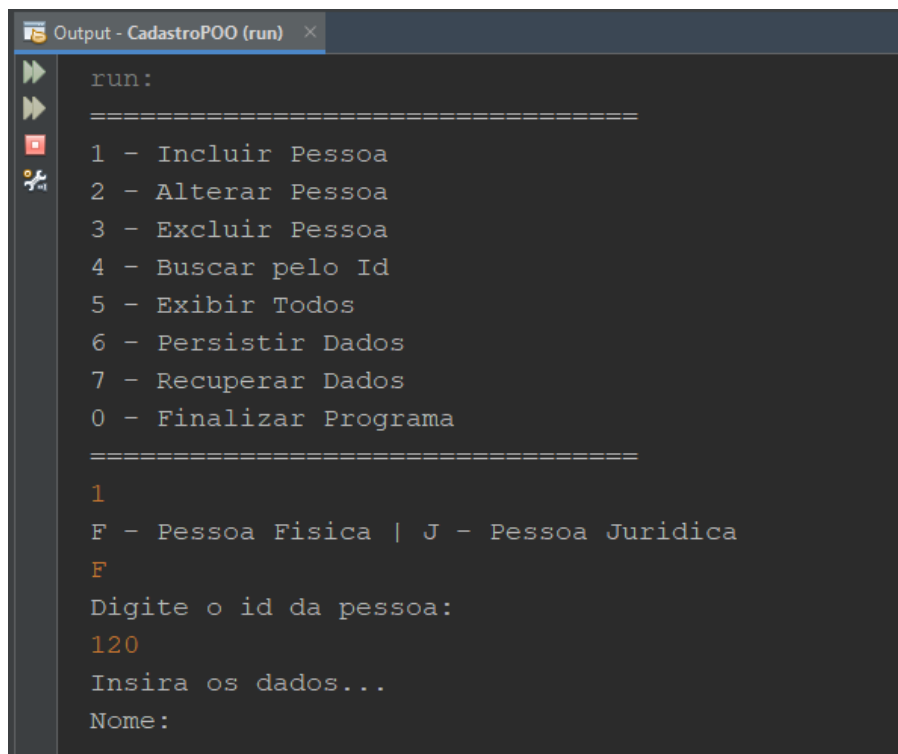
1. Título da Prática;
2. Objetivo da Prática;
3. Todos os códigos solicitados neste roteiro de aula;
4. Os resultados da execução dos códigos também devem ser apresentados;
5. Análise e Conclusão:
  - Quais as vantagens e desvantagens do uso de herança?
  - Por que a interface Serializable é necessária ao efetuar persistência em arquivos binários?
  - Como o paradigma funcional é utilizado pela API stream no Java?
  - Quando trabalhamos com Java, qual padrão de desenvolvimento é adotado na persistência de dados em arquivos?

## 2º Procedimento | Criação do Cadastro em Modo Texto

1. Alterar o método **main** da classe principal do projeto, para implementação do cadastro em modo texto:
  - Apresentar as opções do programa para o usuário, sendo 1 para incluir, 2 para alterar, 3 para excluir, 4 para exibir pelo id, 5 para exibir todos, 6 para salvar dados, 7 para recuperar dados e 0 para finalizar a execução.
  - Selecionada a opção **incluir**, escolher o tipo (Física ou Jurídica), receber os dados a partir do teclado e adicionar no repositório correto.
  - Selecionada a opção **alterar**, escolher o tipo (Física ou Jurídica), receber o **id** a partir do teclado, apresentar os dados atuais, solicitar os novos dados e alterar no repositório correto.

- Seleccionada a opção **excluir**, escolher o tipo (Física ou Jurídica), receber o **id** a partir do teclado e remover do repositório correto.
- Seleccionada a opção **obter**, escolher o tipo (Física ou Jurídica), receber o **id** a partir do teclado e apresentar os dados atuais para a entidade.
- Seleccionada a opção **obterTodos**, escolher o tipo (Física ou Jurídica) e apresentar os dados de todas as entidades do repositório correto.
- Seleccionada a opção **salvar**, solicitar o **prefixo** dos arquivos e persistir os dados nos arquivos **[prefixo].fisica.bin** e **[prefixo].juridica.bin**.
- Seleccionada a opção **recuperar**, solicitar o **prefixo** dos arquivos e obter os dados a partir dos arquivos **[prefixo].fisica.bin** e **[prefixo].juridica.bin**.
- Nas opções **salvar** e **recuperar** devem ser tratadas as exceções.
- Seleccionada a opção **sair**, finalizar a execução do sistema.

2. Ajustar as características para obter uma execução como a seguinte.



```

Output - CadastroPOO (run) x
run:
=====
1 - Incluir Pessoa
2 - Alterar Pessoa
3 - Excluir Pessoa
4 - Buscar pelo Id
5 - Exibir Todos
6 - Persistir Dados
7 - Recuperar Dados
0 - Finalizar Programa
=====
1
F - Pessoa Fisica | J - Pessoa Juridica
F
Digite o id da pessoa:
120
Insira os dados...
Nome:

```

3. Verificar todas as funcionalidades solicitadas e os arquivos gerados.

## Resultados esperados

1. É importante que o código seja organizado.

2. Outro ponto importante é explorar as funcionalidades oferecidas pelo NetBeans para melhoria da produtividade.
3. Nesse exercício, é esperado que o estudante demonstre as habilidades básicas para a programação Java com persistência em arquivo.

### **Relatório discente de acompanhamento**

Os Relatórios de Práticas deverão ser confeccionados em arquivo no formato PDF, com a Logo da Universidade, nome do Campus, nome do Curso, nome da Disciplina, número da Turma, semestre letivo, nome dos integrantes da Prática. Além disso, o projeto deve ser armazenado em um repositório no GIT e o respectivo endereço deve constar na documentação (PDF). A documentação do projeto deve conter:

1. Título da Prática;
2. Objetivo da Prática;
3. Todos os códigos solicitados neste roteiro de aula;
4. Os resultados da execução dos códigos também devem ser apresentados;
5. Análise e Conclusão:
  - O que são elementos estáticos e qual o motivo para o método main adotar esse modificador?
  - Para que serve a classe Scanner?
  - Como o uso de classes de repositório impactou na organização do código?

---

### *Observações*

---

#### **Pré-requisitos:**

1. Os estudantes precisam instalar o JDK e o NetBeans.

#### **Referência Bibliográfica:**

1. <https://stensenme.blob.core.windows.net/hmlgrepoh/00212ti/01703/index.html>
2. <https://stensenme.blob.core.windows.net/hmlgrepoh/00212ti/01338/index.html>
3. <https://stensenme.blob.core.windows.net/hmlgrepoh/00212ti/01662/index.html>

4. Padrões de Projeto. Disponível em <https://refactoring.guru/pt-br/design-patterns>. Acessado em 01/03/2023.
5. Os 4 pilares da Programação Orientada a Objetos. Disponível no endereço <https://www.devmedia.com.br/os-4-pilares-da-programacao-orientada-a-objetos/9264>. Acessado em 01/03/2023.
6. Programação Orientada a Objetos com Java – Dev Media. Disponível em <https://www.devmedia.com.br/programacao-orientada-a-objetos-com-java/18449>. Acessado em 01/03/2023.
7. Tutorial Data Persistence in Java – Code HS. Disponível no endereço <https://codehs.com/tutorial/david/data-persistence-in-java>. Acessado em 01/03/2023.

---

### *Entrega do trabalho prático*

---

**Chegou a hora, gamer!**

 Armazene o projeto em um repositório no GIT.

 Anexar a documentação do projeto (PDF) no GIT.

 Compartilhe o link do repositório do GIT com o seu tutor para correção da prática, por meio da **Sala de Aula Virtual**, na aba "**Trabalhos**" do respectivo nível de conhecimento.

 **Ei, verifique o prazo de entrega pois não aceitamos trabalhos fora do prazo!**